

Look When Unsure, Check When Sure

Consequence training makes a world model’s remaining errors confident—
most of all where it knows the world best

Caio Vicentino
OpenInterpretability
Fortaleza, Brazil
ORCID: 0009-0003-4331-6259
caio@openinterp.org

October 2026

DOI: [10.5281/zenodo.23146971](https://doi.org/10.5281/zenodo.23146971)

Abstract

An agent that can predict the consequences of its actions can chain those predictions and plan without acting, but errors compound, and checking the real state (a `git status`, a sensor read) costs time. We ask when such an agent should look, with V42, the first release candidate of Ekbasis, an open 27B consequence model that reads the probability of every candidate next state from a single forward pass. A simple rule—look when the chain’s confidence since the last look falls below 0.9, plus scheduled checks with a Trickle-style backoff—keeps 197 of 200 fresh 100- and 200-action chains exact in four worlds, two of them never seen in training, at 17.9 looks per 100 actions; never looking keeps 25 of 40 exact. The checks are needed because of a pre-registered finding about the model’s confidence. On 15,008 new questions, 58.1% of its errors in the world families it was trained on carry confidence ≥ 0.9 , against 27.8% in families it never saw (+30.3 points, 95% CI [24.0, 36.5]). The same questions answered by the model *before* consequence training show almost none (1.9% and 0.5%). Training cut its errors in the trained families by two-thirds, but multiplied confident errors per answer tenfold (0.28 to 2.79 per 100), and further training raised them again. In long chains, the parent’s step probabilities never reach 0.9, right or wrong, so the same rule has to look at 99 of every 100 actions: consequence training is what makes the loop cheap, and what makes the checks necessary. Pre-registered fixes that reshape confidence—temperature, Platt and isotonic recalibration, label smoothing, focal loss—fail. Training on the model’s own errors made from true states makes single answers honest, but the loop then looks 81% more. What fixes the loop is mining errors where they happen, inside the model’s own chains (Dagger-style): on fresh chains this cuts silent wrong steps by 71–97% with no more looks. Every such fine-tune, though, failed a pre-registered release criterion, most of them the gate of the git guard that ships with the model, by forgetting rare work-losing commands such as `git reset -merge`. An exact weight interpolation halfway back to V42 passes every single-question and guard criterion, and on 160 fresh chains it keeps the loop’s gain: 91% fewer silent wrong steps with 6% fewer looks. It then passes the pre-registered release evaluation, with the guard judged on the larger set, and is released as Ekbasis-27B, with the checks kept in its default rule. Model, client, data, analysis code and every pre-registration are open.

1 Introduction

Agents act in worlds whose state they rarely see in full. Before running `git checkout - app.py`, a coding agent would like to know whether the command throws away uncommitted work; before a long plan, it would like to know where the plan leaves the world. A *consequence model* (a world model in LeCun’s sense [25], or a forward model in Wolpert’s [51]) answers such questions about the next state given an action. Chained, its predictions let an agent plan without acting. The price is compounding error: a wrong prediction becomes the premise of the next one [46, 17]. Looking at the real state fixes the chain, but every look costs a tool call, a sensor read or a round trip. When should the agent look?

The classical answer is to predict, observe and correct [19], and to observe when the predictor’s own uncertainty crosses a threshold [47, 15]. We apply it to a language world model, V42, the first release candidate of Ekbas-27B, which reads the probability of every answer from one forward pass, with no generation. The chain’s confidence is then the product of those probabilities since the last look, and the agent looks when it drops. On long chains in four worlds, this works (§3). The interesting part is *where it fails*, and why.

The model’s rare errors in the worlds it knows best are confident. A confidence trigger cannot see them, so the default rule adds scheduled checks whose interval doubles while the forecasts hold, as in the Trickle algorithm [40]. A pre-registered test on 15,008 new questions confirms the pattern beyond the chains (§4). A second pre-registered test, against the same model before consequence training, shows that the training made it. Training improved accuracy and, at the same time, made the remaining errors look like right answers.

We then report, in the order we ran them, the attempts to make the model’s confidence honest:

- post-hoc recalibration, training losses and training on the model’s own errors made from true states, which all fail in the loop (§5);
- mining errors inside the model’s own chains, which works, and what it costs (§6).

Every test was pre-registered [32], with its plan’s SHA-256 sealed before the run (Appendix A), unless marked EXPLORATORY.

Contributions.

- **A look-when-unsure loop for a one-pass language world model**, with scheduled checks. It keeps 197 of 200 fresh long chains exact at 17.9 looks per 100 actions, and it looks 2–9× more in worlds the model never saw. It is the default of the released client.
- **Confident errors where the model was trained** (PRE-REGISTERED): 58.1% of errors at ≥ 0.9 in trained families against 27.8% in unseen ones. The ranking of confidences is similar (AUROC 0.92 and 0.91), but the scale is not.
- **Training made it** (PRE-REGISTERED, against the parent model): the difference in gaps is +28.9 points [22.8, 34.9]. Confident errors per answer grew tenfold in trained families while total errors fell by two-thirds. In long chains the parent never reaches 0.9 and has to look at 99 of every 100 actions.
- **What does not fix it** (PRE-REGISTERED): recalibration, label smoothing and focal loss lower the scale but keep the gap, or blur the ranking. Errors mined from true states make single answers honest and the loop look more.

- **What does, and its cost** (PRE-REGISTERED): errors mined inside the model’s own chains cut silent wrong steps in the loop by 71–97% on fresh chains. Every such fine-tune failed a release criterion, most of them the git guard’s, by forgetting rare commands. Halfway back in weight space, the guard holds and the loop keeps its gain (91% fewer silent wrong steps on fresh chains).

2 Setting

Model. V42 is Eikos-27B, a decision model fine-tuned from Qwen3.8-27B [37, 4], further trained with LoRA adapters [16] on consequence questions and merged. It was the first release candidate of Ekbasis-27B; the checkpoint released under that name came out of this work (§6). Given rules, a state, one or more actions and a closed question (yes/no or a choice among listed options), it reads the probability of every option from the next-token distribution over option letters, in one forward pass, at temperature 1. The consequence questions come from two sources:

- synthetic worlds from seven training families (grids, gravity, counters, lamps, sequences, containers, crafting);
- real git repositories: what is in the working tree, the index, the stash and the branches after a command, and whether a command fails or loses uncommitted work.

The second capability ships as a *git guard*, which asks the model before an agent runs a command. Here it doubles as a regression gate for any retraining (§6). The model before consequence training, Eikos-27B, serves as the parent in the causal tests.

Worlds and chains. A chain is 100 or 200 random actions in one of four worlds:

- *containers*: 2–3 containers of 3–10 liters, with fill, empty and pour actions;
- *lamps*: 4–7 lamps with two or three states and 3–4 buttons, each wired to 1–3 lamps, that advance them along their cycle or set them on or off;
- *machines*: 3–5 devices that cycle through 3–4 states, with actions that advance all of them, advance one or reset one;
- *cards*: 4–7 labeled cards in a row, with swaps, segment reversals and shifts.

Containers and lamps belong to the training families. Machines and cards were never trained (0 rows in the training mix).

At each step the model receives the rules, the state it currently believes and one action, and it answers one question per variable of the state (“How many liters are in A?”) in a single request. The next believed state is the most probable answer per variable. For cards it is the most probable *valid* permutation, by assignment over the per-position probabilities [24]. The probability of the state read is the product of the chosen answers’ probabilities, and the *chain confidence* c is the product over the steps since the last look.

A *look* returns the exact real state, which replaces the believed one. A step is a *silent wrong step* when the believed state after it is wrong and no look happened at that step. We report silent wrong steps and looks per 100 actions, and whether the believed state is exactly right after the last action.

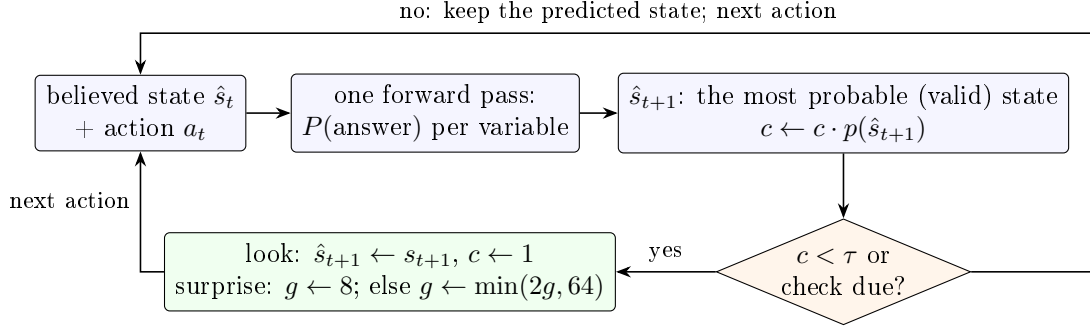


Figure 1: The loop. The model predicts the next state in one forward pass, and the chain confidence c is the product of the probabilities of the states read since the last look. The agent looks at the real state when c falls below τ , or when a scheduled check is due (interval g , Trickle-style backoff). The default is $\tau = 0.9$.

Rules.

- **never**: never look.
- **every 50**: look every 50 actions.
- **step τ** : look when the probability of the state just read is below τ .
- **chain τ** : look when $c < \tau$.
- **until τ** : **chain τ** , and after a look that finds the forecast wrong (a *surprise*), look after every action until a look finds it right.
- **check τ** : **until τ** plus scheduled checks. If g actions pass without a look, look. The interval g starts at 8, doubles (up to 64) after a look that confirms the forecast, and goes back to 8 after a surprise, as in the Trickle algorithm [40].

No rule looks after the last action. The released client’s default is **check 0.9** (Figure 1).

3 Looking when unsure keeps long chains exact

Table 1 compares the rules on the same 40 chains (cards: 8 per length; the other worlds: 4 per length). This is the EXPLORATORY development set on which the default was chosen. Never looking leaves 24 of 40 chains exact: one wrong step corrupts every later one, at 32.45 silent wrong steps per 100 actions. Looking every 50 actions does not help the cards (4 of 16 exact). Confidence-triggered rules keep 39–40 of 40 exact at 9–18 looks per 100 actions.

Two things separate the rules:

- **The trigger must accumulate.** **step 0.9** misses errors that build up over several confident steps (7 of 8 lamp chains exact); the chain trigger does not.
- **Checks halve the silent wrong steps at a small cost.** **chain 0.9** leaves 0.73 silent wrong steps per 100 actions, **check 0.9** 0.40, for 1.4 more looks per 100 actions.

Looking again after a surprise until right (**until**) changes nothing on its own; the checks are what helps, and §4 shows why.

Table 1: Rules compared on the same 40 chains (100 and 200 actions; cards 16, the other worlds 8 each). Silent wrong steps and looks are per 100 actions. Right: looks per 100 actions by world.

rule	exact	silent wrong	looks	containers	lamps	machines	cards
never	24/40	32.45	0	0	0	0	0
step 0.9	39/40	2.15	12.0	1.0	0.0	1.7	28.8
chain 0.5	39/40	2.82	8.7	0.2	0.2	2.1	20.5
until 0.5	39/40	2.55	9.6	0.3	0.2	2.1	22.7
check 0.5	40/40	1.30	11.1	3.2	2.9	3.7	22.8
chain 0.9	40/40	0.73	16.7	1.6	3.3	11.8	33.4
until 0.9	40/40	0.80	17.6	2.1	3.3	12.2	35.1
check 0.9 (default)	40/40	0.40	18.1	3.8	4.4	12.2	35.1
<i>cards only (16 chains)</i>							
every 50	4/16	46.54	1.3				
chain 0.9	16/16	0.00	33.4				
two views + chain 0.9	16/16	0.00	17.0				

It looks where it is unsure. Under the default, the model looks at containers and lamps 3.8 and 4.4 times per 100 actions, at machines 12.2 and at cards 35.1. The two worlds it never saw get 3–9 $\times$ more looks, and all their chains stay exact. Cards are the hardest: a permutation of 4–7 labels, read position by position. There a second view helps. Asking also the inverse questions (“At which position is the X?”, a question type the model knows) and reading both views jointly halves the looks (17.0 against 33.4 per 100 actions), with no error. In an EXPLORATORY end-to-end check through the released client (4 card chains of 100 actions), the same option cut looks from 44.5 to 27.5 per 100 actions with all chains exact.

Fresh chains. On 200 chains never used to design the rule (80 card chains, 40 per other world; seeds fixed in the plans of later pre-registered comparisons), the default keeps 197 of 200 exact, at 0.64 silent wrong steps and 17.9 looks per 100 actions:

	containers	lamps	machines	cards	all
exact	40/40	40/40	40/40	77/80	197/200
silent wrong per 100 actions	1.68	1.48	0.00	0.03	0.64
looks per 100 actions	4.1	4.8	10.2	35.3	17.9

Each of the three misses is a 200-action card chain whose only silent wrong step is the last action, which no rule looks after. The model had flagged it: the probability of the state read was 0.0005, 0.002 and 0.025. On 40 of these chains, never looking keeps 25 exact.

4 Where the errors are confident

The observation (EXPLORATORY). In 40 never-look chains (6,000 steps) we recorded each step’s probability and whether the step itself was right from the model’s own previous state:

- containers: all 5 wrong steps at probability 0.917–0.988;
- lamps: all 4 at 0.991–0.994;
- machines: 1 of 3 wrong steps at ≥ 0.9 ;

Table 2: Confident errors on 15,008 new questions (PRE-REGISTERED), for V42 and for the same model before consequence training (Eikos-27B, the parent), on the same items through the same service at temperature 1. T: trained families; H: trained families, held-out question types; U: families never trained.

		n	wrong (% of answers)	errors at ≥ 0.9 (% of errors)	confident errors per 100 answers	median confidence of the errors	AUROC
V42	T	7,172	4.8	58.1	2.79	0.936	0.922
	H	2,472	13.5	37.1	5.02	0.791	0.888
	U	5,364	14.9	27.8	4.14	0.748	0.908
Eikos-27B (before)	T	7,172	14.4	1.9	0.28	0.568	0.901
	H	2,472	35.2	0.7	0.24	0.467	0.924
	U	5,364	21.3	0.5	0.11	0.539	0.900

- cards: 0 of 82 (the highest 0.626).

In the never-seen worlds the model errs often and says so. In the trained worlds it errs rarely and does not. No single per-step threshold serves all four worlds, and the confidence trigger alone lets the trained worlds’ errors through. The checks catch them: in containers, silent wrong steps go from 2.58 to 1.17 per 100 actions (Table 1, **chain** 0.9 against **check** 0.9). Nine errors on the trained side is too few for a claim, and card orderings differ from the other worlds in kind. So we tested it.

The test (PRE-REGISTERED). We generated 15,008 new short questions (1–3 actions) from the world generator:

- 12 families, 120 per family, question type and number of actions, with seeds never used before;
- three groups: **T**, the 7 training families with their trained question types (7,172); **H**, the same families with a held-out question type (2,472); **U**, 5 families never trained (5,364).

The primary statistic is the share of wrong answers made with confidence ≥ 0.9 , T against U. Its 95% interval comes from a percentile bootstrap that resamples items within each family (2,000 resamples) [3].

Confirmed (Table 2, Figure 2). 58.1% of the errors in T carry confidence ≥ 0.9 , against 27.8% in U: +30.3 points, 95% CI [24.0, 36.5]. The family medians agree (60.9% against 25.8%). Among answers that the actions change (secondary), the gap is larger: 62.0% against 21.6%, +40.4 [32.8, 48.1]. H lies in between (37.1%).

The AUROC of confidence for right against wrong answers is similar in T and U (0.922 and 0.908): the model *ranks* its answers about as well in both groups, but the *scale* differs. One consequence matters for agents. A trigger at 0.9 can flag at most 41.9% of the errors in T, against 72.2% in U. Per answer, though, U still has more confident errors (4.14 against 2.79 per 100), because it has three times as many errors.

Robustness (EXPLORATORY).

- The pattern holds for yes/no questions (61.3% against 41.2%) and choices (55.6% against 19.9%).
- It holds in every difficulty stratum.
- It holds when any single family is left out (gap 22.2–33.5 points).

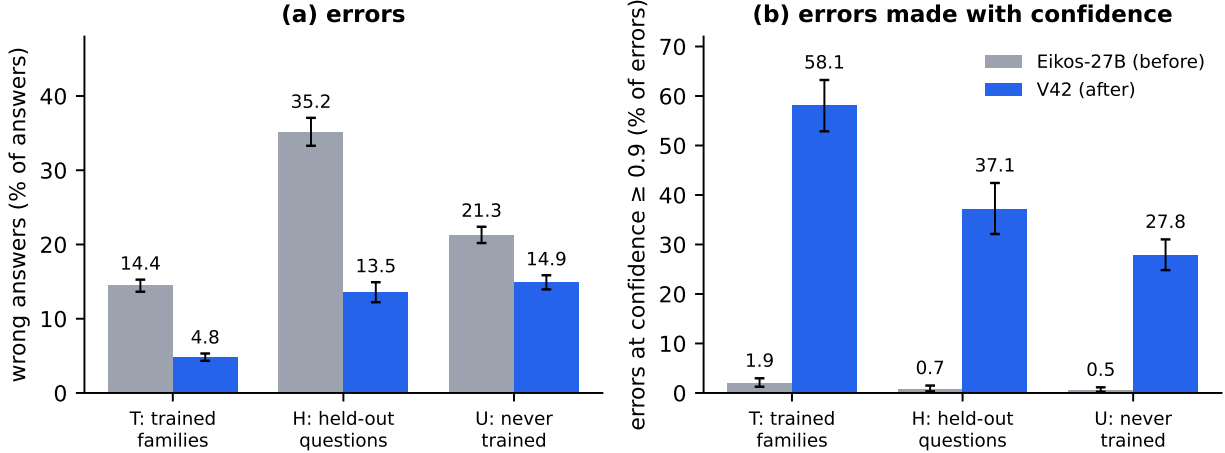


Figure 2: Consequence training cut errors (a) and made the remaining ones confident (b), most of all in the families it trained on. The same 15,008 questions, the parent model (Eikos-27B) and V42. Error bars: Wilson 95% intervals, for display; the tests use the pre-registered bootstraps.

A logistic regression on the wrong answers gives familiarity an odds ratio of 2.44, controlling for the cell’s error rate and the answer format. With families as the unit (a family-cluster bootstrap), however, the interval includes zero ($[-0.35, 1.63]$ on the log-odds), because two families go against the trend: timers (U, 61.8%) and counters (T, 32.4%). It is a strong tendency across items, not a law across families.

Training made it (PRE-REGISTERED). Was the gap there before consequence training, for instance because the unseen families are simply harder? We asked the parent model, Eikos-27B, the same 15,008 questions through the same service, letter readout and temperature. Its errors are frequent but almost never confident: 1.9% of its errors in T and 0.5% in U are at ≥ 0.9 (gap +1.4, $[0.4, 2.4]$). Its confidence never exceeds 0.976, a ceiling of its letter readout, but 0.9 sits inside its range: 54.7% of its right answers reach it, against 92.6% of V42’s. The pre-registered statistic, the difference between the two models’ gaps by a paired bootstrap, is $D = +28.9$ points, $[22.8, 34.9]$. The training made the gap.

What the training did is visible in Table 2:

- it cut errors by two-thirds in T (14.4% to 4.8%) and by under a third in U (21.3% to 14.9%);
- it raised the share of confident errors by 56.2 points in T $[50.8, 61.5]$ and 27.3 in U $[24.3, 30.4]$;
- per answer, confident errors went from 0.28 to 2.79 per 100 in T, and from 0.11 to 4.14 in U.

On each model’s own scale (EXPLORATORY; the threshold at the confidence of its 10th-percentile right answer: 0.608 for the parent, 0.943 for V42), the parent already shows a small gap in the same direction (+8.8 points) against +28.3 after training. Familiar worlds were slightly more prone to confident errors from the start, and the training amplified it and lifted the whole scale.

More training, more of it (EXPLORATORY). The control arm of our first repair round (§5) continued V42’s recipe for 200 more steps. It raised the share of confident errors further, from 57.7% to 67.3% in T and from 27.1% to 36.0% in U (the trainer’s own readout), without improving

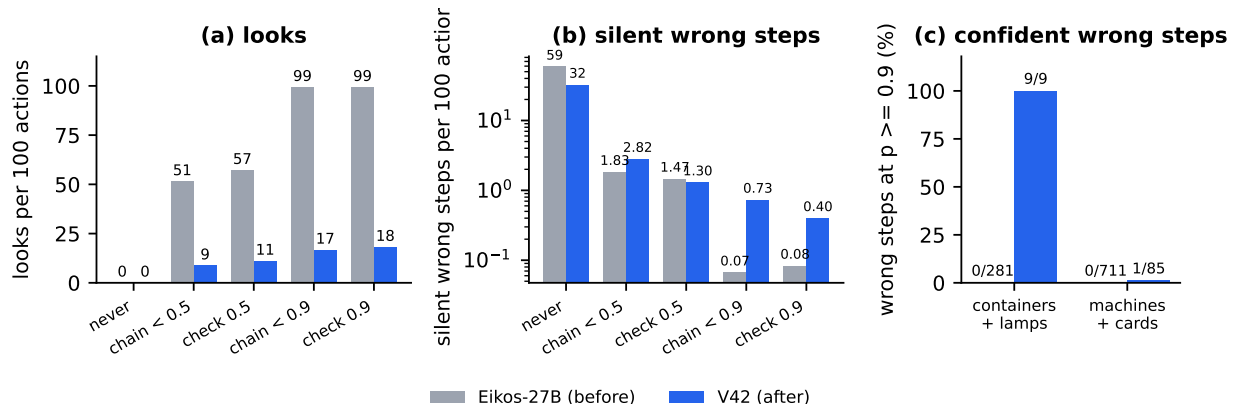


Figure 3: The same 40 chains before and after consequence training (PRE-REGISTERED). (a) Looks and (b) silent wrong steps per 100 actions under each rule. (c) The share of the never-look traces’ wrong steps made at step probability ≥ 0.9 , with the counts above the bars. The parent errs far more often but never with confidence, so it looks at nearly every action. V42 looks at fewer than a fifth as many and needs the checks for its confident errors.

accuracy. Guo et al. [10] saw networks grow overconfident late in training, as they overfit the log-likelihood while their classification error still fell. Here it grows where the model has seen the most.

In the loop (PRE-REGISTERED). We ran the parent on the 40 chains of Table 1, with every rule (Figure 3).

- **The prediction holds.** In its never-look traces of the two worlds V42 trained on, 0 of the parent’s 281 wrong steps reached 0.9, against all 9 of V42’s (one-sided Fisher $p = 2.8 \times 10^{-17}$).
- **It holds for a simpler reason than on single questions.** The parent’s step probabilities never reach 0.9 at all, right or wrong; its highest is 0.8755. A step’s probability is the product of several answers, each below 0.976. They still rank its right steps above its wrong ones (AUROC 0.961 in the trained worlds, 0.914 in the unseen ones).
- **It looks at almost every action.** The parent is wrong 31 times as often per step in the trained worlds (11.7% against 0.375%). It runs the loop with 99.3 looks per 100 actions at chain confidence 0.9 and 51.5 at 0.5, against 16.7 and 8.7 for V42.
- **Its checks add nothing:** 0.07 and 0.08 silent wrong steps per 100 actions, without and with them. V42’s checks halve its silent wrong steps.
- **Its misses are flagged.** It ends 35 or 36 of 40 chains exact at 0.9. Each of the 9 chains that end wrong missed only its last action, which no rule looks after, at a probability of at most 0.38.

Consequence training is what makes the loop cheap, and what makes the checks necessary.

5 What does not fix it

Post-hoc recalibration (PRE-REGISTERED). We fit four maps on 4,998 new items (seeds never used; 40 per family, question type and number of actions) plus the git validation split (774):

- M1, a global temperature;
- M2, a temperature from features of the question;
- M3, Platt scaling of the chosen answer’s confidence [34];
- M4, isotonic regression [53].

We then tested them on the 15,008 questions and the git splits (Table 3, top).

- **The band becomes honest.** The 0.9–0.99 band, which says 96.7% but is right 84.0% of the time at $T=1$, is right 96.8–97.1% after any map. ECE [31] falls from 0.049 to 0.010–0.027.
- **The gap stays.** The share of errors at ≥ 0.9 falls in T and in U alike (T 59.1% to 24–32%, U 27.7% to 6–10%), so the T–U gap remains (18–22 points): a map from confidence to confidence cannot tell a familiar confident error from a familiar right answer.
- **The guard fails.** Every map raised the guard’s false alarms on unseen git commands beyond the pre-registered margin (2.4% to 4.8%; 11.9% for isotonic).

None was adopted.

Training losses (PRE-REGISTERED). From the V42 adapter, 200 more steps of V42’s recipe with:

- cross-entropy (the control);
- label smoothing 0.1 [44, 29];
- focal loss with $\gamma=2$ [26, 28].

Label smoothing cut the confident share to 26.8% / 14.7% (T/U) but lowered the AUROC from 0.919 to 0.871. Focal loss cut it to 9.3% / 1.5% with accuracy kept, but lowered the AUROC to 0.891 and became under-confident: its 0.9–0.99 band said 94.2% and was right 99.1% of the time. Losses that only lower the scale blur the ranking a trigger depends on.

Training on its own errors, made from true states (PRE-REGISTERED). V42 answered 47,758 new questions from the 7 trained families and was wrong on 2,486 (5.2%), 1,381 of them at ≥ 0.9 .

- **r2err**, trained on those errors (30% of every batch, 300 steps), made single answers honest. Confident errors per 100 answers fell from 2.76 to 0.22 in T (–92%), and also in the held-out types and the never-trained families, which were never mined (H 4.81 to 0.44, U 4.03 to 0.56). The AUROC rose from 0.919 to 0.928.
- In the loop, on the same 40 chains, r2err looked 81% more (32.8 against 18.1 per 100 actions): it had become unsure where it was right. Right answers at ≥ 0.99 fell from 74.4% to 44.1%.
- **r3w20** (§6, Table 4) mixed in right-but-unsure answers (2,486 errors and 4,972 right answers below 0.99, 20% of every batch). It beat V42 on the 15,008 questions (accuracy 90.5% against 90.2%; 1.37 confident errors per 100 answers in T against 2.76) and kept most of its sharpness (68.0% of right answers at ≥ 0.99 , against 74.4%). In the loop, on 240 paired chains, it was not better: 0.50 against 0.42 silent wrong steps and 23.1 against 18.9 looks per 100 actions. In the trained worlds’ never-look traces, 16 of its 23 wrong steps (in 4,800 steps) were confident, against 11 of 13 (in 2,400) for V42.

Table 3: What did not fix it: post-hoc recalibration (PRE-REGISTERED; V42’s answers, fit on separate items) and training losses (PRE-REGISTERED; 200 more steps from the V42 adapter, the trainer’s readout). “Errors at ≥ 0.9 ”: share of the errors, T / U.

recalibration	ECE	0.9–0.99 band right	errors at ≥ 0.9 (T / U)	guard false alarms (unseen git)
none ($T=1$)	0.049	84.0%	59.1 / 27.7	2.4%
M1 global temperature	0.027	96.8%	31.9 / 10.4	4.8%
M2 temperature from features	0.027	97.0%	29.8 / 9.9	4.8%
M3 Platt	0.019	97.1%	27.5 / 7.6	4.8%
M4 isotonic	0.010	97.0%	24.3 / 6.4	11.9%
training loss	AUROC	error in T	errors at ≥ 0.9 (T / U)	
V42	0.919	4.78%	57.7 / 27.1	
cross-entropy, 200 more steps (control)	0.916	5.41%	67.3 / 36.0	
label smoothing 0.1	0.871	5.09%	26.8 / 14.7	
focal loss, $\gamma=2$	0.891	4.67%	9.3 / 1.5	

The errors a model makes from true states are not the errors it makes in a chain, where it reads states it predicted itself. This is the exposure problem of sequence prediction [2, 39], and the reason DAgger trains on the learner’s own distribution [41].

Changing the loop instead (PRE-REGISTERED). Without training, on the 40 chains of Table 1:

- looking also after the last action: 0.50 silent wrong steps and 18.2 looks, against 0.40 and 18.1;
- a threshold that adapts to surprises: 1.03 and 13.1, with 39 of 40 exact;
- a second view of each state (“is A full?”, “are lamps 1 and 2 in the same state?”) that triggers a look when it disagrees: 0.17 and 26.5 (silent wrong steps -58% , looks $+46\%$);
- only confident disagreements: 0.23 and 25.1;
- fusing the two views into one read, by exact Viterbi/forward over the neighbors: 0.82 and 16.4;
- partial looks at the least sure variables only (containers, lamps and machines, 24 chains): 1.17 silent wrong steps against 0.67, for 13% fewer variables seen.

None improved the default. A second view helps only when it is as reliable as the first and its errors are independent, which is the case of the cards’ inverse questions (§3) and not of the derived “same state?” questions. A whole look also corrects, in passing, the confident errors of variables the trigger did not suspect, and a partial look gives that up.

6 Mining errors where they happen, and what it costs

Chain mining. V42 ran 200 new chains in the two trained worlds (containers and lamps; seeds used by no test), never looking. At every step, each variable it answered wrong, or right below 0.99, was saved with the true result of the action *from the state it was reading*, its own prediction. This gave 555 items (134 wrong, 421 right but unsure). Machines and cards were never mined. The idea is DAgger’s [41], and Data as Demonstrator’s for multi-step prediction [48]: train on the states the model itself visits. The runs are in Table 4.

Table 4: Training on its own errors. Single questions: the trainer’s readout on the 15,008 items (accuracy; confident errors per 100 answers in T / H / U). Loop: the default rule on chains paired with V42, silent wrong steps and looks per 100 actions. Guard: the pre-registered gate on the larger guard set (Table 5).

run (from V42)	accuracy	confident errors	loop: silent wrong, looks	guard gate
V42	90.2	2.76 / 4.81 / 4.03	—	—
r2err: errors from true states (30%)	89.4	0.22 / 0.44 / 0.56	0.40→0.02, 18.1→32.8	not run
r3w20: errors + unsure answers from true states (20%)	90.5	1.37 / 2.39 / 3.00	0.42→0.50, 18.9→23.1	not run
r4a : errors + unsure (20%) + chain items (10%)	90.7	1.70 / 3.20 / 3.37	0.93→0.03 , 18.6→18.2	fails (recall)
r4b : r3w20, then errors + unsure (10%) + chain items (15%)	90.8	1.85 / 4.05 / 4.23	0.93→0.09 , 18.6→15.0	fails (recall)
r4c : chain items only (20%)	90.3	2.34 / 4.49 / 4.25	0.45→0.13 , 16.4→12.9	fails (alarms)
r5a-c: chain items, git share of every batch kept	89.8–91.0	1.92–3.03 (T)	not run	2 of 3 fail
w4a5 : halfway between V42 and r4a	90.7	2.05 / 3.84 / 3.67	0.57→0.05 , 16.7→15.7	passes

It fixes the loop. An EXPLORATORY test on the 240 chains of the earlier loop tests came first: silent wrong steps fell from 0.42 to 0.07 (r4a) and 0.13 (r4b) per 100 actions, at 18.7 and 15.1 looks against 18.9. A pre-registered confirmation on 160 fresh chains then confirmed both (Figure 4b):

- r4a: silent wrong steps 0.93 to 0.03 per 100 actions (−97%), looks 18.6 to 18.2;
- r4b: 0.93 to 0.09 (−90%), looks 15.0 (−19%), with 79 exact chains against 77 of 80.

The gains reach the worlds that were never mined. In machines, looks fell from 9.6 to 4.5 (r4a) and 3.5 (r4b) per 100 actions with no wrong step, and r4b kept 31 of 32 card chains exact against 29. Lamps went from 3.04 silent wrong steps per 100 actions to none in both runs.

r4c, trained on the chain items alone, was confirmed on another 160 fresh chains: 0.45 to 0.13 (−71%), looks 16.4 to 12.9 (−21%), 80 of 80 exact. Training on errors from true states pushed the loop toward more looks, while chain mining moved it to fewer silent errors at the same or lower cost (Figure 4b).

It costs rare knowledge. A release candidate must also pass the git guard’s pre-registered gate:

- work-losing commands flagged at $p \geq 0.2$, and false alarms on safe ones, on unseen repositories;
- each within one point of V42, on a split of known command types and one of command types held out of training.

V42 flags 50 of 51 and 41 of 42 work-losing commands, with 0 of 51 and 1 of 42 false alarms. The results:

- r4a flagged 39 of 42 held-out commands (2 of 42 false alarms);
- r4b flagged 49 of 51 known and 37 of 42 held-out commands;
- r4c flagged all 42 held-out commands, but missed one known command and raised more false alarms (1 of 51 known, 3 of 42 held-out);

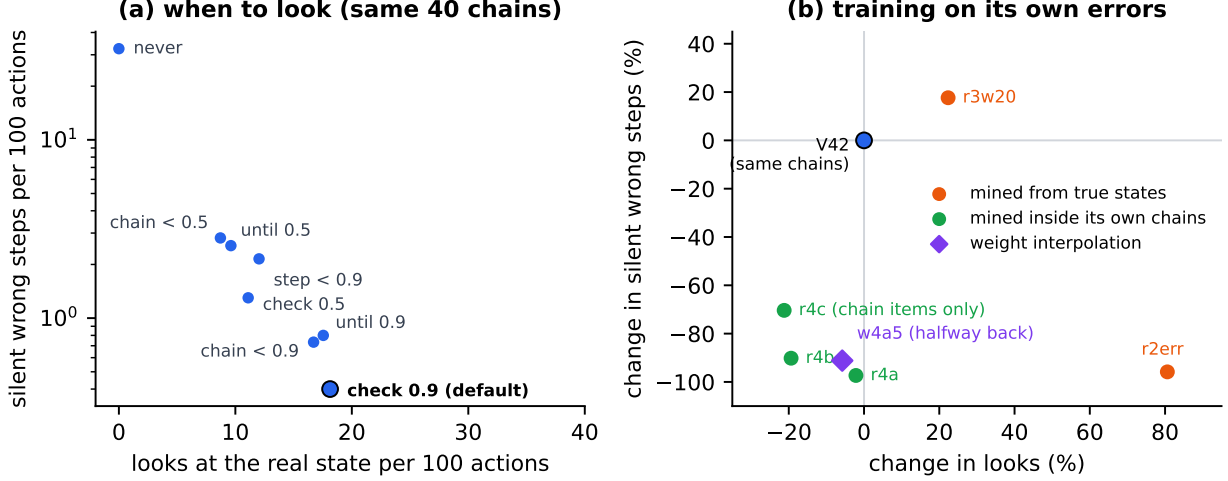


Figure 4: (a) The rules of Table 1 on the same 40 chains (V42); the default is circled. (b) Training on the model’s own errors, against V42 on the same chains (default rule): errors mined from true states (orange) buy honesty with looks; errors mined inside the model’s own chains (green) cut silent wrong steps with no more looks; the weight interpolation (purple) keeps that gain. r2err: 40 chains; r3w20: 240; r4a and r4b: 160 fresh; r4c and w4a5: another 160 fresh each.

- a fifth round kept every batch’s git share at V42’s, and all three runs still lost two or three held-out commands (the trainer’s readout: 38, 39 and 38 of 42, against V42’s 41).

What was lost is specific. r4a stopped flagging two work-losing command sequences and r4b five, five distinct ones in all. Three of them involve `git reset -merge`, and both runs missed two of those; the other two are a `git stash drop` after a `stash pop` and a `git restore -source=HEAD~1`. Meanwhile the median probability they gave to work-losing commands *rose* (0.983 to 0.986 and 0.991). This is forgetting of rare cases [22, 27], not a shift of the whole scale.

A larger gate. On the small sets, one item is worth more than the one-point margin. We therefore pre-registered a 17× larger guard set, drawn from repository states that no earlier set contains:

- known command types: 863 work-losing and 863 safe commands;
- command types held out of training: 799 work-losing and 799 safe commands.

It replaced the small sets as the gate, with the same rule of one point per split, and every model was scored by the trainer’s readout (Table 5). That readout matches the served model: through vLLM and the API, the release model flags 827 and 771 work-losing commands and raises 50 and 54 false alarms, and it makes the same flag decision as the readout on 3,321 of the 3,324 items.

- Three of the six chain-mined runs lose work-losing commands beyond the margin on both splits (r4a, r4b and r5c), and r5a on the held-out split.
- r4c flags slightly more work-losing commands than V42, but raises too many false alarms (59 and 66 against 50 and 53).
- r5b passes the guard, but had already failed another criterion of its round: answers that the actions change in multi-step questions, 80.3% against 81.3%.

Halfway back in weight space. Interpolating the weights of a fine-tuned model with those of its starting point can keep most of the gains and recover what was lost [52]. With LoRA adapters the interpolation is exact: the two adapters are concatenated into one of rank 128, with the factors scaled by $\sqrt{1-\lambda}$ and $\sqrt{\lambda}$, which gives the weights $W_0 + (1-\lambda)\Delta_{\text{release}} + \lambda\Delta_{\text{run}}$. We pre-registered three interpolations.

- **w4a5**, halfway between V42 and r4a ($\lambda = 0.5$), passes every criterion: the guard on both splits, every single-question criterion, and confident errors in T not higher (2.05 against 2.76 per 100 answers). Its accuracy on the 15,008 questions is 90.7% against 90.2%, and 74.1% of its right answers stay at ≥ 0.99 , against 74.4%.
- w4a7 ($\lambda = 0.7$ toward r4a) and w4b5 (halfway toward r4b) lose too many known work-losing commands.

In the loop (PRE-REGISTERED). w4a5 then ran on 160 chains that no model or test had used (cards seeds 48–63, the other worlds 24–31), paired with V42, under round 4’s confirmation criteria. It is confirmed:

- silent wrong steps 0.57 to 0.05 per 100 actions (−91%), looks 16.7 to 15.7 (−6%), and 79 of 80 chains exact for both;
- in containers, where V42’s confident errors concentrate, 2.79 to 0.21 silent wrong steps per 100 actions;
- in machines, never mined, looks 9.9 to 6.8.

Interpolating halfway back kept nearly all of r4a’s gain in the loop and none of its losses on the guard.

Release evaluation (PRE-REGISTERED). On the exact merged weights, served:

- every non-guard criterion passes: git3 96.62% of all questions against 96.39%; answers that the actions change in multi-step questions 81.16% against 81.16% (78.03% against 78.20% read once); the never-trained families’ changed answers 81.88% against 81.72%; ECE 0.024 against 0.027;
- the 16 fresh real-repository scenarios are identical: 3 of 3 work-losing flagged, 1 false alarm of 13;
- on the small git sets it flags 40 of the 42 held-out work-losing commands, against 41. The original gate would fail it by that one item;
- under the gate that replaced it (`PLAN_guard_set.md`), it passes. Served on the larger set, it flags 828 and 767 work-losing commands (against 827 and 771) and raises 37 and 49 false alarms (against 50 and 54);
- on the real-use test, rerun on the two repositories’ later state (72 new scenarios, 28 work-losing), it and V42 answer alike: 28 of 28 flagged, no false alarm, 92 of 96 “will it fail?” right;
- under the injection secondary, one more borderline answer crossed 0.5: a “will it fail?” that the planted branch name moved from 0.44 to 0.53 (V42: 0.13 to 0.27). No work-losing command was missed.

It is the first candidate to pass every gate.

Table 5: The git guard on the larger pre-registered set (PRE-REGISTERED; the trainer’s readout). Work-losing commands flagged at $p \geq 0.2$ and false alarms on safe commands, on known command types (863 + 863) and held-out ones (799 + 799). The gate is one point per split against V42.

model	known types		held-out types		gate
	flagged	false alarms	flagged	false alarms	
V42	829	50	771	53	—
r4a	812	44	763	49	fails
r4b	776	27	732	35	fails
r4c	832	59	776	66	fails
r5a	825	45	758	45	fails
r5b	821	33	764	44	passes
r5c	820	37	759	30	fails
w4a5	826	40	768	49	passes
w4a7	819	38	769	47	fails
w4b5	819	36	766	45	fails

Real use (EXPLORATORY). We ran 72 scenarios on independent copies of two real repositories in their current state (remotes pointed at local paths, hooks off). Truth comes from executing each scenario on its copy: 28 scenarios lose uncommitted work, and 9 contain a command that fails. V42 and r4c gave the same answers:

- all 28 work-losing scenarios flagged, with no false alarm on the other 44;
- 92 of 96 “will it fail?” questions right;
- a median of 0.89 s per check.

Whatever separates the two models on the generated sets did not show up in these 72 scenarios.

The release decision. The interpolation was the first candidate to pass every gate, and it is released as Ekbasis-27B, with **check** 0.9 kept as the client default: its familiar errors are fewer, not gone. A retraining of the whole lineage with chain items from the start, where the rare git knowledge is learned alongside them rather than overwritten, remains the cleaner route.

7 Discussion

Why familiar errors are confident. One reading fits every measurement here, though we did not test it causally. Consequence training lifts the whole confidence scale.

- Before it, 54.7% of the parent’s right answers reach 0.9. After it, 92.6% do, and 74.4% reach 0.99 (83.8% in the trained families).
- In the worlds the model trained on, its few remaining errors come out of that sharpened distribution and look like its right answers.
- In worlds it never saw, the scale is lower for right and wrong answers alike (62.4% of right answers at ≥ 0.99 in U), so more of its errors stay below the trigger.

The ranking survives (AUROC 0.922 and 0.908), so what fails is a fixed threshold, not the model’s ordering. The chains say the same: the parent ranks its right steps above its wrong ones about as well as V42 (AUROC 0.961 and 0.946 in the trained worlds), on a scale that never reaches 0.9. Further training moves in the same direction, and on each model’s own scale the gap is larger after training than before (+28.3 against +8.8 points).

What to do in an agent.

- Trigger looks on the chain’s accumulated confidence, not on single steps.
- Add scheduled checks with a backoff. Here they cost 1.4 looks per 100 actions and halve the silent wrong steps.
- Expect the trigger to fire least where the model is most trusted: the checks are for that part.
- Use a second view only if it is as reliable as the first and fails independently, as the cards’ inverse questions do. Otherwise it buys honesty with looks.
- To train the problem away, mine errors on the states the model itself visits.
- Gate every retraining on rare cases with a set large enough that one item is not the whole margin.

What the next test caught. Several results that looked like fixes did not survive the next test.

- r3w20 beat V42 on the 15,008 questions and not in the loop.
- r2hard had no silent wrong step on the 100-action chains (against 0.55 per 100 actions), but on the 200-action chains it had as many (0.35 against 0.33), while looking more at both lengths.
- The chain-mined runs that won the loop then lost rare git commands.

A single test would have reported each of these as a fix. Read together, they show the pattern: the gain follows where the training data comes from, and something else gives way.

8 Related work

When to observe.

- *Predict, observe, correct* is Kalman’s [19].
- *Event-triggered sampling and estimation* measure when needed rather than on a clock [1, 45].
- *Variance-based triggering* measures when the estimator’s own uncertainty crosses a threshold, and its schedules tend to become periodic [47].
- *Active observing in continuous-time control* learns a model that observes when its uncertainty crosses a threshold, and shows that regular observation is not optimal [15].
- *Infoprop* tracks the error accumulated along model-based rollouts [6].
- *Trickle* gives the backoff our checks use [40].

Our loop is this principle applied to a language world model whose uncertainty is read from logits in one pass. We add a measured failure mode of the trigger, confident errors where the model was trained, which is why scheduled checks belong in the default.

Language agents that consult the world. FLARE retrieves when a language model is unsure [5]; KnowNo asks for help from multiple-choice likelihoods [23]. Song and Cai probe the environment and report that self-reported uncertainty fails under confident wrong beliefs [43], which matches what we measure in the familiar part of the model’s world. Language world models predict the consequences of actions for planning:

- reasoning as planning with a world model [13];
- text-based world simulators [49];
- web world models [9];
- Qwen-AgentWorld, which reasons at length before predicting [38]; Ekbas answers in one pass.

In learned-model RL [11, 12, 30], compounding model error limits rollout length [46, 17]. Closest for agents:

- Active Epistemic Control queries the environment to verify uncertain facts and uses world-model predictions only to prune plans [35];
- DreamPhase imagines futures offline and scores them with an uncertainty-aware value [36];
- guards that predict the consequence or risk of an agent’s action before it runs, on the web [50] and on mobile interfaces, where SeerGuard predicts the next screen [42].

Our git guard is of the last kind, and here it also serves as a regression gate.

Calibration and familiarity.

- Modern networks are overconfident [10], and calibration degrades under dataset shift [33].
- The maximum softmax probability separates misclassified and out-of-distribution inputs from correct ones [14], and selective prediction builds on it [7, 20].
- Language models mostly know what they know [18], post-training can hurt calibration [8], and how fine-tuning treats unfamiliar examples shapes how models err [21].

Our result sits between these. V42’s errors in unfamiliar worlds are frequent and flagged, while its rare errors in familiar worlds are confident, and the parent model shows that consequence training created the difference.

Training on the learner’s own states, and forgetting.

- DAgger [41] and Data as Demonstrator [48] train on the learner’s own distribution; scheduled sampling [2] and sequence-level training [39] address exposure bias.
- Fine-tuning forgets [22, 27]; weight-space interpolation can trade the gains against the losses [52].

9 Limitations

- **Synthetic chain worlds.** The four chain worlds are synthetic, and a look is exact and costs the same every time. The real-repository evidence (the git guard) concerns single commands, where the agent can read `git status` before each one; a long-horizon loop in a real environment is the next test.
- **One model.** We study one model lineage (a 27B hybrid model) and one parent, so whether other consequence models show the same familiarity effect is open.
- **Small numbers in the chain observation.** It rests on 9 trained-world errors in the traces. The pre-registered tests carry the claim, and at the family level their effect is a tendency (two families go against it).
- **The parent’s ceiling.** The parent’s letter readout never puts more than 0.976 on one option, so a fixed threshold compares scales as much as models. The own-scale analysis and the AUROCs address this, but only in part.
- **Different readouts.** The trainer’s readout (training rounds) and the service’s readout (release tests) differ slightly, for instance 57.7% against 58.1% for V42 in T; each comparison uses one readout on both sides.
- **Training data.** The replay data of the lineage came from the parent’s training snapshot and included about 48 draws of FinQA items, a family the parent uses for evaluation. No evaluation here uses FinQA.

10 Conclusion

A one-pass consequence model can keep long action chains exact by looking only when its chain confidence drops. It looks rarely in the worlds it knows and often in those it does not. Its confidence fails in a specific place, though: where it was trained, its few remaining errors look like its right answers. Training made them so, and more training makes more of them. Recalibration and smoother losses do not remove them; training on the errors it makes inside its own chains does, at the price of rare knowledge elsewhere, and an exact interpolation halfway back to V42 keeps the gain without the price. Its familiar errors are fewer, not gone: an agent should still look when unsure and check when sure.

Acknowledgments

This work was done with Claude Opus 5.5 (Anthropic), which helped plan and run the experiments, write the analysis code and the text, and check every number and reference.

Reproducibility

Everything below is open:

- the released model, the interpolation (`caiovicentino1/Ekbasis-27B`, with FP8, INT4 and MLX builds), and V42’s predictions and chains, the data of this paper (`results_v42/` in the code repository);

- the client with the loop (`ekbasis, simulate(observe=...)`);
- the training data (`caiovicentino1/ekbasis-data`).

The code, with the saved predictions and chains, and the model weights are under Apache-2.0. In the training data, the generated rows (worlds and git) are under Apache-2.0, and the replayed rows keep their sources' licenses (CC BY 4.0, ODC-BY or MIT; each row names its own, and the dataset card lists them). This paper is under CC BY 4.0. The paper folder of the code repository holds:

- every analysis script;
- `reproduce.py`, which re-runs each pre-registered analysis on the saved predictions and chains and checks that it reproduces the stored result exactly;
- `numbers.json`, with every number cited here, and `check_numbers.py`, which checks that every number in the text comes from it;
- `make_figs.py`.

Every plan's SHA-256 was sealed before its run (Appendix A); `reproduce.py` also checks that every plan still matches its seal.

A Pre-registrations

Each plan was written and its SHA-256 recorded (`PREREG_sha256.txt`) before the run it governs. The plans are published with the code: the `PREREG` files at the repository root and the `PLAN` files under `paper/prereg/`. Times are BRT (UTC−3).

plan	what it tests	sealed
<code>PREREG_confident_errors.md</code>	T against U, 15,008 questions	10-03 18:30, 9644dbbe
<code>PREREG_confident_errors_eikos.md</code>	the same on the parent; D	10-03 18:48, f5d00582
<code>PREREG_recalibration.md</code>	M1–M4	10-03 19:11, 0dede309
<code>PLAN_v43_calibration.md</code>	training losses	10-03 19:11, 9fee0b87
<code>PLAN_v43_round2.md</code>	errors from true states	10-03 19:57, de7b86d0
<code>PLAN_v43_loop_test.md</code>	r2err in the loop	10-03 21:39, 9d69d449
<code>PLAN_v43_round3.md</code>	dose; r3w20 in the loop	10-03 22:48, 9b1e422e
<code>PLAN_loop_ideas.md</code> , 2, 3, 4	loop changes without training	10-04 00:55–03:14
<code>PLAN_v43_round4.md</code>	chain mining	10-04 00:55, f5a43813
<code>PLAN_v43_round4_loop_explore.md</code>	r4a, r4b on 240 chains (EXPLORATORY)	10-04 03:04, 95069db2
<code>PLAN_v43_round4_confirm.md</code> , _r4a.md	confirmation on 160 fresh chains	10-04 04:36, 1988728f; 04:38, 5089819e
<code>PLAN_v43_release_eval.md</code>	the release gate	10-04 04:41, 6f98d66a
<code>PLAN_v43_round5.md</code>	git share kept	10-04 07:51, a865a3fe
<code>PLAN_v43_r4c_loop.md</code>	r4c on 160 fresh chains	10-04 08:55, 1f7d1809
<code>PLAN_v43_wise.md</code> , addendum	weight interpolation	10-04 08:57, 1f82e2a0; 09:44, 8666e4d0
<code>PLAN_guard_set.md</code>	the $17\times$ larger guard set	10-04 09:42, 88c71c83
<code>PLAN_paper_base_loop.md</code>	the parent in the loop	10-04 13:02, 0b4ba207

References

- [1] K. J. Åström and B. M. Bernhardsson. Comparison of Riemann and Lebesgue Sampling for First Order Stochastic Systems. In Proceedings of the 41st IEEE Conference on Decision and Control (CDC), vol. 2, pp. 2011–2016, 2002. DOI: 10.1109/CDC.2002.1184824.

- [2] S. Bengio et al. Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks. In Advances in Neural Information Processing Systems 28 (NIPS), 2015. arXiv:1506.03099.
- [3] B. Efron. Bootstrap Methods: Another Look at the Jackknife. The Annals of Statistics, 7(1):1–26, 1979. DOI: 10.1214/aos/1176344552.
- [4] C. Vicentino. Eikos-27B (model) and Eikos (code). Hugging Face and GitHub, 2026. <https://huggingface.co/caiovicentino1/Eikos-27B>.
- [5] Z. Jiang et al. Active Retrieval Augmented Generation. In Proceedings of EMNLP, 2023. arXiv:2305.06983.
- [6] B. Frauenknecht et al. On Rollouts in Model-Based Reinforcement Learning. In International Conference on Learning Representations (ICLR), 2025.
- [7] Y. Geifman and R. El-Yaniv. Selective Classification for Deep Neural Networks. In Advances in Neural Information Processing Systems 30 (NIPS), 2017. arXiv:1705.08500.
- [8] OpenAI. GPT-4 Technical Report. arXiv preprint, 2023. arXiv:2303.08774 (Figure 8: post-training hurts calibration).
- [9] Y. Gu et al. Is Your LLM Secretly a World Model of the Internet? Model-Based Planning for Web Agents. Transactions on Machine Learning Research (TMLR), 2025. arXiv:2411.06559.
- [10] C. Guo et al. On Calibration of Modern Neural Networks. In Proceedings of the 34th International Conference on Machine Learning (ICML), PMLR 70:1321–1330, 2017. arXiv:1706.04599.
- [11] D. Ha and J. Schmidhuber. Recurrent World Models Facilitate Policy Evolution. In Advances in Neural Information Processing Systems 31 (NeurIPS), 2018. arXiv:1809.01999.
- [12] D. Hafner et al. Dream to Control: Learning Behaviors by Latent Imagination. In International Conference on Learning Representations (ICLR), 2020. arXiv:1912.01603.
- [13] S. Hao et al. Reasoning with Language Model is Planning with World Model. In Proceedings of EMNLP, pp. 8154–8173, 2023. DOI: 10.18653/v1/2023.emnlp-main.507.
- [14] D. Hendrycks and K. Gimpel. A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks. In International Conference on Learning Representations (ICLR), 2017. arXiv:1610.02136.
- [15] S. Holt, A. Hüyük, and M. van der Schaar. Active Observing in Continuous-time Control. In Advances in Neural Information Processing Systems 36 (NeurIPS), 2023.
- [16] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. In International Conference on Learning Representations (ICLR), 2022. arXiv:2106.09685.
- [17] M. Janner et al. When to Trust Your Model: Model-Based Policy Optimization. In Advances in Neural Information Processing Systems 32 (NeurIPS), 2019. arXiv:1906.08253.
- [18] S. Kadavath et al. Language Models (Mostly) Know What They Know. arXiv preprint, 2022. arXiv:2207.05221.
- [19] R. E. Kalman. A New Approach to Linear Filtering and Prediction Problems. Journal of Basic Engineering, 82(1):35–45, 1960. DOI: 10.1115/1.3662552.
- [20] A. Kamath, R. Jia, and P. Liang. Selective Question Answering under Domain Shift. In Proceedings of ACL, pp. 5684–5696, 2020. DOI: 10.18653/v1/2020.acl-main.503.
- [21] K. Kang et al. Unfamiliar Finetuning Examples Control How Language Models Hallucinate. In Proceedings of NAACL (Long Papers), pp. 3600–3612, 2025. DOI: 10.18653/v1/2025.naacl-long.183.

- [22] J. Kirkpatrick et al. Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. DOI: 10.1073/pnas.1611835114.
- [23] A. Z. Ren et al. Robots That Ask For Help: Uncertainty Alignment for Large Language Model Planners. In *Conference on Robot Learning (CoRL)*, 2023. arXiv:2307.01928.
- [24] H. W. Kuhn. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955. DOI: 10.1002/nav.3800020109.
- [25] Y. LeCun. A Path Towards Autonomous Machine Intelligence, version 0.9.2. OpenReview, 2022. <https://openreview.net/forum?id=BZ5a1r-kVsf>.
- [26] T.-Y. Lin et al. Focal Loss for Dense Object Detection. In *Proceedings of ICCV*, pp. 2999–3007, 2017. DOI: 10.1109/ICCV.2017.324.
- [27] Y. Luo et al. An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-Tuning. *IEEE Transactions on Audio, Speech and Language Processing*, 33:3776–3786, 2025. DOI: 10.1109/TASLPRO.2025.3606231.
- [28] J. Mukhoti et al. Calibrating Deep Neural Networks using Focal Loss. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, pp. 15288–15299, 2020. arXiv:2002.09437.
- [29] R. Müller, S. Kornblith, and G. E. Hinton. When Does Label Smoothing Help? In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. arXiv:1906.02629.
- [30] J. Schrittwieser et al. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. *Nature*, 588:604–609, 2020. DOI: 10.1038/s41586-020-03051-4.
- [31] M. P. Naeini, G. F. Cooper, and M. Hauskrecht. Obtaining Well Calibrated Probabilities Using Bayesian Binning. In *Proceedings of AAAI*, 29(1):2901–2907, 2015. DOI: 10.1609/aaai.v29i1.9602.
- [32] B. A. Nosek et al. The Preregistration Revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. DOI: 10.1073/pnas.1708274114.
- [33] Y. Ovadia et al. Can You Trust Your Model’s Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. arXiv:1906.02530.
- [34] J. C. Platt. Probabilities for SV Machines. In A. J. Smola, P. Bartlett, B. Schölkopf, and D. Schuurmans, editors, *Advances in Large-Margin Classifiers*, pp. 61–74. MIT Press, 2000. DOI: 10.7551/mitpress/1113.003.0008.
- [35] S. Qu. Active Epistemic Control for Query-Efficient Verified Planning. arXiv preprint, 2026. arXiv:2602.03974.
- [36] S. Mohajer Hamidi, L. Ye, and K. N. Plataniotis. DreamPhase: Offline Imagination and Uncertainty-Guided Planning for Large-Language-Model Agents. In *International Conference on Learning Representations (ICLR)*, 2026.
- [37] Qwen Team. Qwen3.8-27B. Hugging Face model card, 2026. <https://huggingface.co/Qwen/Qwen3.8-27B>.
- [38] Y. Zuo et al. Qwen-AgentWorld: Language World Models for General Agents. arXiv preprint, 2026. arXiv:2606.24597.
- [39] M. Ranzato et al. Sequence Level Training with Recurrent Neural Networks. In *International Conference on Learning Representations (ICLR)*, 2016. arXiv:1511.06732.
- [40] P. Levis, T. Clausen, J. Hui, O. Gnawali, and J. Ko. The Trickle Algorithm. RFC 6206, IETF, 2011.

- [41] S. Ross, G. J. Gordon, and J. A. Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In *Proceedings of AISTATS*, PMLR 15:627–635, 2011. arXiv:1011.0686.
- [42] X. Yu et al. SeerGuard: A Safety Framework for Mobile GUI Agents via World Model Prediction. arXiv preprint, 2026. arXiv:2607.15550.
- [43] X. Song and Z. Cai. Ask the World Before Acting: Environment Probing for Calibrated Agent World Models. arXiv preprint, 2026. arXiv:2606.31422.
- [44] C. Szegedy et al. Rethinking the Inception Architecture for Computer Vision. In *Proceedings of CVPR*, pp. 2818–2826, 2016. DOI: 10.1109/CVPR.2016.308.
- [45] P. Tabuada. Event-Triggered Real-Time Scheduling of Stabilizing Control Tasks. *IEEE Transactions on Automatic Control*, 52(9):1680–1685, 2007. DOI: 10.1109/TAC.2007.904277.
- [46] E. Talvitie. Model Regularization for Stable Sample Rollouts. In *Proceedings of the 30th Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 780–789, 2014.
- [47] S. Trimpe and R. D’Andrea. Event-Based State Estimation With Variance-Based Triggering. *IEEE Transactions on Automatic Control*, 59(12):3266–3281, 2014.
- [48] A. Venkatraman, M. Hebert, and J. A. Bagnell. Improving Multi-Step Prediction of Learned Time Series Models. In *Proceedings of AAAI*, 29(1):3024–3030, 2015. DOI: 10.1609/aaai.v29i1.9590.
- [49] R. Wang et al. Can Language Models Serve as Text-Based World Simulators? In *Proceedings of ACL (Short Papers)*, pp. 1–17, 2024. DOI: 10.18653/v1/2024.acl-short.1.
- [50] B. Zheng et al. WebGuard: Building a Generalizable Guardrail for Web Agents. arXiv preprint, 2025. arXiv:2507.14293.
- [51] D. M. Wolpert, Z. Ghahramani, and M. I. Jordan. An Internal Model for Sensorimotor Integration. *Science*, 269(5232):1880–1882, 1995. DOI: 10.1126/science.7569931.
- [52] M. Wortsman et al. Robust Fine-Tuning of Zero-Shot Models. In *Proceedings of CVPR*, pp. 7949–7961, 2022. DOI: 10.1109/CVPR52688.2022.00780.
- [53] B. Zadrozny and C. Elkan. Transforming Classifier Scores into Accurate Multiclass Probability Estimates. In *Proceedings of KDD*, pp. 694–699, 2002. DOI: 10.1145/775047.775151.